

CSC 220 — Reference Reading

Arrays, ArrayList, LinkedList, and the `List` Interface

1. Why Do We Need Data Structures?

Programs frequently need to store and manage a **collection of related data**.

For example, suppose we have several `Student` objects:

```
Student s1 = new Student("Alice", 1, 2.5f);
Student s2 = new Student("Bob", 2, 3.5f);
Student s3 = new Student("Alison", 3, 1.5f);
```

We could create a separate variable for every student, but this quickly becomes inconvenient.

Instead, Java provides different ways of organizing collections of data. In this lesson, we examine:

- Arrays
- `ArrayList`
- `LinkedList`
- the `List` interface

The important question is not simply:

Which one is better?

Instead, we should ask:

Which data structure is appropriate for the operations our program needs to perform?

2. Arrays

An array stores a fixed number of elements.

```
int[] x = new int[3];

x[0] = 1;
x[1] = 2;
x[2] = 3;
```

We can visualize this as:

Index:	0	1	2
x	1 2 3		

An important property of an array is that its **size is fixed when the array is created**.

Therefore:

```
x[3] = 23;
```

causes an error because index 3 does not exist.

If we need more space, we must create another array:

```
int[] y = new int[10];

for (int i = 0; i < x.length; i++) {
    y[i] = x[i];
}

y[3] = 23;
```

The original array `x` did not become larger. We created a **different array** and copied the existing values into it.

This raises an important question:

Wouldn't it be useful if Java could manage this resizing for us?

That is one reason collection classes such as `ArrayList` are useful.

3. Arrays Can Store Objects

Arrays are not limited to primitive values.

For example:

```
Student[] students = new Student[3];

students[0] = new Student("Alice", 123, 4.0f);
students[1] = new Student("Bob", 113, 4.0f);
students[2] = new Student("Blake", 101, 4.0f);
```

We can traverse the array using an enhanced `for` loop:

```
for (Student student : students) {  
    System.out.println(student);  
}
```

The number of positions in an array is available through:

```
students.length
```

Remember that `length` is **not a method** for arrays.

4. ArrayList

An `ArrayList` is a Java collection class that behaves like a **resizable array**.

First import it:

```
import java.util.ArrayList;
```

Then we can create one:

```
ArrayList<Student> students = new ArrayList<>();
```

For now, interpret:

```
<Student>
```

as:

This collection is intended to contain `Student` objects.

The `<Student>` syntax is related to **Generics**, which we will formally study later.

5. Adding Elements to an ArrayList

We can add objects using `add()`:

```
Student bob = new Student("Bob", 222, 3.97f);  
  
ArrayList<Student> students = new ArrayList<>();  
  
students.add(bob);  
students.add(new Student("Bobby", 21102, 3.17f));  
students.add(new Student("Allison", 1133, 2.7f));
```

Unlike a regular array, we did not have to specify the final size in advance.

The `ArrayList` manages its capacity internally.

6. Traversing an ArrayList

The enhanced `for` loop works with an `ArrayList`:

```
for (Student student : students) {  
    System.out.println(student);  
}
```

This should look very similar to traversing an array.

7. ArrayList `size()`

For an array:

```
students.length
```

For an `ArrayList`:

```
students.size()
```

Example:

```
System.out.println(students.size());
```

Notice:

```
Array          → length
```

```
ArrayList      → size()
```

8. Accessing an Element: `get()`

With an array, we use:

```
students[0]
```

With an `ArrayList`, we use:

```
students.get(0);
```

Example:

```
Student firstStudent = students.get(0);  
System.out.println(firstStudent);
```

Indexes still begin at 0.

9. Replacing an Element: `set()`

Suppose we want to replace the object stored at index 1.

For an array:

```
students[1] = new Student(...);
```

For an `ArrayList`:

```
students.set(  
    1,  
    new Student("Charlie", 555, 3.5f)  
);
```

`set()` **replaces** the existing element.

10. `set()` VS. `add(index, object)`

These operations should not be confused.

Consider:

```
students.set(1, newStudent);
```

This means:

Replace the object currently at index 1.

However:

```
students.add(1, newStudent);
```

means:

Insert the new object at index 1.

The existing elements are moved to make room.

For example:

Before:

0	1	2
Alice	Bob	Alison

```
students.add(1, David);
```

After:

0	1	2	3
Alice	David	Bob	Alison

Bob was not replaced. Bob moved from index 1 to index 2.

11. Removing Elements from an ArrayList

`ArrayList` provides two important overloaded versions of `remove()`.

Remove using an index

```
Student removedStudent = students.remove(1);
```

This removes the element at index 1.

The method returns the object that was removed.

Remove using an object

```
boolean result = students.remove(bob);
```

This attempts to remove the specified object.

It returns:

true → the object was found and removed

false → the object was not found

Therefore, these overloaded methods have different return types:

```
E remove(int index)
```

```
boolean remove(Object o)
```

12. Why Can ArrayList Insertion Be Expensive?

An `ArrayList` is backed internally by an array.

Conceptually:

```
[ Alice ][ Bob ][ Alison ][ Ali ]  
    0         1         2         3
```

Suppose we insert David at index 0.

Before:

```
[ Alice ][ Bob ][ Alison ][ Ali ]
```

After:

```
[ David ][ Alice ][ Bob ][ Alison ][ Ali ]
```

The existing elements have to move.

Conceptually:

```
Alice →  
Bob    →  
Alison →  
Ali    →
```

Therefore, inserting or removing elements near the beginning or middle of a large `ArrayList` may require many elements to shift.

However, accessing an element using its index is very efficient:

```
students.get(500);
```

The underlying array structure makes indexed access efficient.

13. LinkedList

Java also provides a `LinkedList`.

Import it using:

```
import java.util.LinkedList;
```

Example:

```
LinkedList<Student> students = new LinkedList<>();

students.add(new Student("Alice", 1, 2.5f));
students.add(new Student("Bob", 2, 3.5f));
students.add(new Student("Alison", 3, 1.5f));
students.add(new Student("Ali", 4, 2.5f));
```

A `LinkedList` can also grow and shrink dynamically.

However, its internal organization is very different from an `ArrayList`.

14. Think of a LinkedList as a Chain

Conceptually, imagine:

```
Alice ⇄ Bob ⇄ Alison ⇄ Ali
```

The items do not need to occupy neighboring locations in memory.

Instead, the list consists of **nodes connected using references**.

Java's `LinkedList` is a **doubly linked list**.

Conceptually, a node contains:

```
+-----+
| previous | element | next      |
+-----+
```

Therefore, one node can refer to both its previous and next neighbors.

A simplified picture is:

```
previous      next
  ←  [ Bob ]  →
```

The important idea is:

An `ArrayList` organizes its element references using an underlying array, while a `LinkedList` organizes its elements through connected nodes.

15. Basic LinkedList Operations

Many operations look familiar:

```
students.add(student);
students.add(2, student);
students.get(2);
students.remove(2);
students.size();
```

This is intentional. `ArrayList` and `LinkedList` provide many of the same operations.

16. Operations at the Beginning and End

`LinkedList` provides convenient operations for working with its two ends.

Add to the beginning

```
students.addFirst(student);
```

Add to the end

```
students.addLast(student);
```

Retrieve the first element

```
Student first = students.getFirst();
```

Retrieve the last element

```
Student last = students.getLast();
```

Remove the first element

```
Student removed = students.removeFirst();
```

Remove the last element

```
Student removed = students.removeLast();
```

17. `add(0, student)` VS. `addFirst(student)`

For a `LinkedList`, these produce the same resulting ordering:

```
students.add(0, student);
```

and:

```
students.addFirst(student);
```

Both place the new student at the beginning.

However:

```
addFirst(student)
```

more clearly communicates the intention:

Add this element to the **front** of the linked list.

18. `LinkedList` and Index Access

A `LinkedList` supports:

```
students.get(2);
```

but its internal structure is not an array.

Conceptually:

Alice ⇄ Bob ⇄ Alison ⇄ Ali

To find an arbitrary position, Java may need to follow links through the structure.

Therefore, frequent random/index-based access is generally not a strength of a `LinkedList`.

This is an important distinction:

`ArrayList`

↓

Very good at accessing an element by index.

`LinkedList`

↓

May need to travel through connected nodes to locate an arbitrary position.

19. Inserting and Removing in a `LinkedList`

Suppose we have:

Alice ⇄ Bob ⇄ Alison ⇄ Ali

and want to insert Charlie between Bob and Alison.

Conceptually:

Alice ⇄ Bob ⇄ Charlie ⇄ Alison ⇄ Ali

The surrounding links can be changed.

We do **not** need to shift a block of elements as an `ArrayList` would.

However, there is an important qualification:

Java must still locate the requested position.

Therefore, it would be incorrect to say:

“`LinkedList` insertion is always faster.”

The cost depends on the operation and whether the required location must first be found.

20. What About Memory?

A `LinkedList` is generally **not more memory-efficient** than an `ArrayList`.

An `ArrayList` conceptually maintains an array of references:

```
[ ref ][ ref ][ ref ][ ref ][ empty ][ empty ]
  ↓      ↓      ↓      ↓
Alice   Bob   Alison Ali
```

There may be some unused capacity.

A `LinkedList`, however, needs a separate node for each element:

```
+-----+
| previous | Student ref | next |
+-----+
```

Each node requires additional references to its neighbors.

Therefore, a `LinkedList` typically has **more per-element memory overhead**.

So we should not think:

`LinkedList` is better because it saves memory.

That is generally not its advantage.

21. ArrayList vs. LinkedList

Characteristic	ArrayList	LinkedList
Size	Dynamic	Dynamic
Internal idea	Resizable array	Connected nodes
Indexed access	Fast	Generally slower
<code>get(index)</code>	Efficient	May require traversal
Add at end	Usually efficient	Efficient
Insert/remove near beginning	May require shifting	Links can be changed without shifting

Characteristic	ArrayList	LinkedList
Insert/remove at arbitrary index	May require shifting	No shifting, but location must first be found
Memory overhead	Generally lower	Generally higher
Elements/nodes need to be adjacent?	Underlying reference array is contiguous	Nodes do not

The key lesson is:

Neither data structure is universally better.

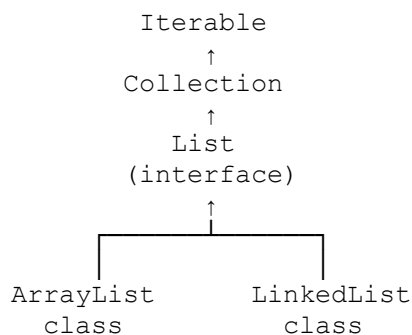
The appropriate choice depends on what operations the program performs frequently.

22. The `List` Interface

Why do `ArrayList` and `LinkedList` have so many similar methods?

Because both belong to Java's `List` abstraction.

A simplified view is:



`List` is an **interface**.

`ArrayList` and `LinkedList` are **classes** that implement the `List` interface.

The interface defines common list behavior.

This includes operations such as:

```

add()
get()
set()
remove()

```

```
size()
```

23. Connecting This to Polymorphism

We have already discussed the difference between a **declared type** and an **actual type**.

Consider:

```
List<Student> students = new ArrayList<>();
```

Here:

Declared type	Actual object type
↓	↓

```
List<Student> students = new ArrayList<>();
```

We could instead write:

```
List<Student> students = new LinkedList<>();
```

Many operations can remain exactly the same:

```
students.add(new Student("Alice", 1, 3.5f));
students.add(new Student("Bob", 2, 3.7f));

System.out.println(students.get(0));
System.out.println(students.size());
```

This is another practical example of **polymorphism**.

Our code can depend on the `List` interface rather than being tightly tied to one particular implementation.

Remember to import `List`:

```
import java.util.List;
```

24. Invalid Indexes

Arrays and list classes require valid indexes.

For example:

```
students.get(100);
```

will fail if the list does not contain an element at index 100.

This results in an:

`IndexOutOfBoundsException`

This connects today's lesson with our previous discussion of Java exceptions.

25. Quick Reference

Array

```
Student[] students = new Student[3];  
  
students[0] = student;  
  
Student s = students[0];  
  
int size = students.length;
```

ArrayList

```
ArrayList<Student> students = new ArrayList<>();  
  
students.add(student);  
students.add(index, student);  
  
Student s = students.get(index);  
  
students.set(index, student);  
  
students.remove(index);  
students.remove(student);  
  
int size = students.size();
```

LinkedList

```
LinkedList<Student> students = new LinkedList<>();  
  
students.add(student);  
students.add(index, student);  
  
students.addFirst(student);  
students.addLast(student);  
  
Student s = students.get(index);  
  
Student first = students.getFirst();
```

```
Student last = students.getLast();

students.remove(index);
students.removeFirst();
students.removeLast();

int size = students.size();
```

Using the `List` Interface

```
List<Student> students = new ArrayList<>();
```

or:

```
List<Student> students = new LinkedList<>();
```

26. Main Takeaways

Array

An array provides a fixed-size collection with efficient indexed access.

```
[ A ] [ B ] [ C ] [ D ]
  0      1      2      3
```

Its major limitation is that its size cannot change after creation.

ArrayList

An `ArrayList` provides a **dynamic/resizable array-like structure**.

```
[ A ] [ B ] [ C ] [ D ] [   ] [   ]
```

It is particularly useful when we want dynamic size while still performing frequent indexed access.

Insertion and removal in the beginning or middle may require elements to shift.

LinkedList

A `LinkedList` organizes elements as **connected nodes**.

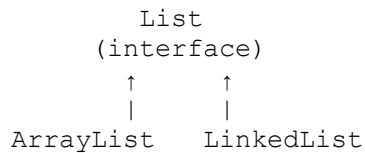
```
A ⇌ B ⇌ C ⇌ D
```

The nodes do not need to sit next to one another in memory. They are connected using references.

Changing links does not require shifting a block of elements, but locating an arbitrary position may require traversal through the list.

List

`List` is an interface representing ordered list behavior.



This allows us to write code such as:

```
List<Student> students = new ArrayList<>();
```

and later potentially use another `List` implementation:

```
List<Student> students = new LinkedList<>();
```

This is an application of interfaces and polymorphism.

Final Thought

When choosing a data structure, do not ask:

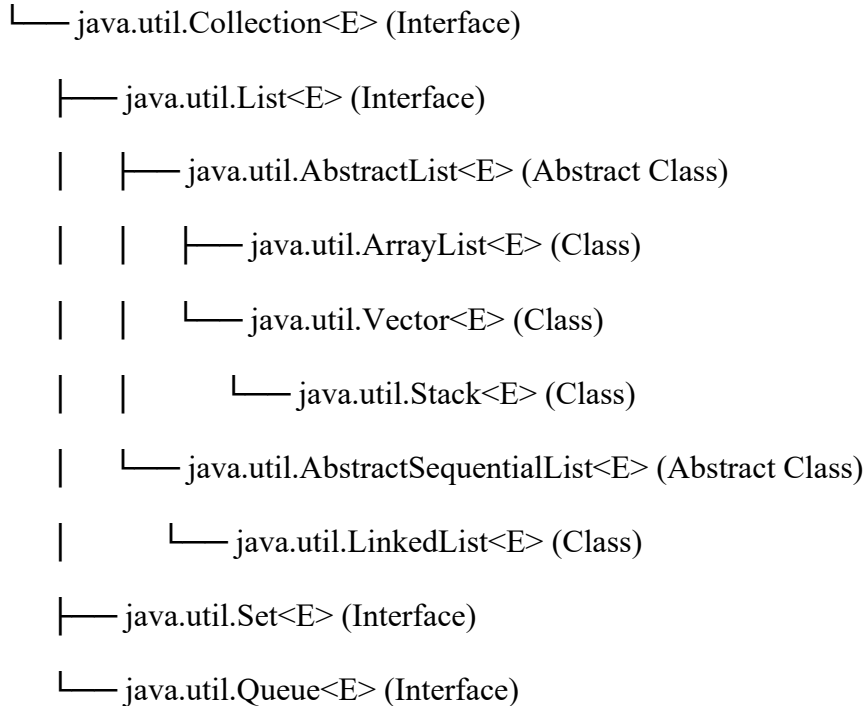
“Which data structure is the best?”

Instead ask:

“What operations will my program perform frequently?”

That question helps us choose an appropriate data structure.

java.lang.Iterable<T> (Interface)



- **Iterable:** The top-level interface containing the `iterator()` method, enabling use in `for-each` loops. [\[1\]](#), [\[2\]](#), [\[3\]](#), [\[4\]](#)
- **Collection:** The root interface for storing groups of objects, extending `Iterable`. [\[1\]](#), [\[2\]](#)
- **List:** An ordered sub-interface of `Collection` that allows duplicate elements. [\[1\]](#), [\[2\]](#)
- **AbstractList:** A skeletal abstract class implementing `List` to reduce effort in creating custom list data structures. [\[1\]](#)
- **ArrayList & LinkedList:** Concrete implementation classes used in everyday development for resizable arrays and doubly-linked lists.